

precept-research: ground-up status, concepts and execution plan

Gian Prem Rajaram · UCL MSc CS · Supervisors Prof. Jun Wang and Prof. Philip Treleaven · 25 July 2026

Repository state at commit d31f159 (PR #44, 'p0-p1 fixes'). Private operating manual, not a submission artefact.

1. Bottom line up front

The engineering is done and correct; the science has not started. 19 of 30 tickets are merged, the full simulator-to-CPVI-to-gate spine carries 197 tests across 35 files with CI enforcing ruff, mypy --strict, an 80 per cent coverage gate scoped to the load-bearing core and a bandit scan, and the last merge closed two P0 construct defects that would have invalidated the headline result. Zero episodes have been recorded and zero datasets exist. The binding constraint is not Myriad access; it is that (1) the Phase-1 pilot has never run, and (2) there is no driver script that calls the pilot, so even with a served model today you would have to write about thirty lines before anything executes.

What the last push actually bought you is the ability to make real progress without Myriad. Three things now hold: the whole pipeline points at any OpenAI-compatible endpoint through one config field (`base_url`) with no code change; the manifest permanently stamps which substrate produced each dataset, so interim data can never be confused with cluster data; and a set of CPU-only diagnostics (feasibility search, episode renderer, transcript dumper, shuffled-message audit) let you validate the task and the estimator with no GPU and no LLM at all. The feasibility tool has already paid for itself - it proved the shipped difficulty ladder was geometrically impossible, which would have failed the pilot gates for the wrong reason and burned the Myriad window on an uninterpretable null.

The two-week priority is therefore: rent one GPU, run the pilot, freeze Y and V. Everything downstream of the pilot verdict either depends on its findings or is cheap once it lands. The one genuinely buildable-ahead item that needs neither compute nor pilot results is RQ3a (DSE-023/024), and it is blocked not on code but on an unwritten methodological decision: what the outcome variable Y is on real failure logs.

2. What we are doing, from the ground up

Six links in one chain. Each link is a claim the thesis has to defend, and each maps to a specific module in the repo.

(1) Why split a task across agents at all. The failure literature is brutal about multi-agent LLM systems: Cemri et al. report 41 to 86.7 per cent task-failure rates across seven frameworks (1,600+ traces, kappa 0.88), and the 2026 reliability-limits result proves formally that a delegated network sharing the same evidence is dominated by a single centralised decision maker with that evidence. So the only defensible reason to split is that the task does not fit in one context window. Splitting is not free, and the cost is paid at the boundary where one agent compresses what it knows into a message.

(2) The boundary is the failure site, and it is unmeasured. Inter-agent misalignment is roughly a third of observed failures. The field can label a trace as failed and can try, badly, to attribute a failure to a step - the best published Who&When method reaches 53.5 per cent on the responsible agent and 14.2 per cent on the decisive step. Nobody has a per-handoff score computable at the moment of the handoff. That absence is the gap this dissertation occupies.

(3) Why V-usable information rather than mutual information. The ideal quantity is the conditional information a message carries about the outcome. It is not estimable at sentence-embedding dimensionality; nearest-neighbour estimators break past roughly ten dimensions and our vectors are 768. V-information (Xu et al.) fixes this by making the measure model-relative: it asks how much a message helps a specified bounded predictive family predict the outcome. It is zero when the family cannot extract the signal, which is exactly the right honesty for language agents - a message carries usable information only if the receiver can read it. Ethayarajh et al. make it pointwise, so you get a score per handoff rather than per corpus.

(4) Why conditioned, and on whose state. When two agents look at the same board, a probe that appears to read the message may just be reading state both already had. Hewitt et al.'s conditional probing subtracts a state-only baseline so only information beyond that baseline counts. The decision now pre-registered - and this is the single most important construct choice in the project - is that the conditioning state s is the state observable to the receiver, not the sender. Under C3 (asymmetric visibility) that is the windowed view. Before the last merge the code conditioned on the sender's full state, which made C3 arithmetically inert: the baseline probe already knew the goal, so a perfectly informative message would have scored roughly zero added bits.

(5) Why a physics task. The outcome Y has to be a real event, not a scripted label, or CPVI is measuring an annotation. A T-shaped load is pushed through three chambers joined by two narrow slits in a deterministic Pymunk arena - a two-agent transposition of Dreyer et al.'s ant-versus-human cooperative-transport puzzle, where restricting human communication made groups perform worse than a single individual. Success, progress and collision are read off the simulator.

(6) Why a gate, and why it cannot threshold CPVI. Lowe et al. showed that correlation between a message statistic and an outcome is not evidence the message caused the outcome; only manipulating the channel settles it. So we block low-information handoffs at runtime and compare against a matched-firing-rate control and a random-trigger control. But CPVI needs the realised outcome, so it cannot be thresholded live. The gate thresholds a target-free statistic computable at the handoff, calibrated offline against realised failure and never against CPVI. Calibrating against CPVI is the circularity error, and s_cos , a probe-independent cosine, exists specifically to answer that objection.

CPVI in one line. Fit two probes on frozen embeddings: g_cond on [receiver-observed state ; message] and g_base on [receiver-observed state], both predicting Y . Per handoff, $CPVI = \log_2 g_cond(y)$ minus $\log_2 g_base(y)$. Always report unconditional PVI alongside and the PVI minus CPVI gap - that gap is how much apparent message value was an echo of state the receiver already held.

3. What is built, module by module

Signals: the whole spine from arena to gate calibration exists and is tested.

Abbreviations: CPVI = conditional pointwise V-usable information; PVI = its unconditional twin; Y = the outcome a probe predicts; V = the probe model family; COM = centre of mass; AUROC = area under the ROC curve; ECE = expected calibration error; DSE-*nnn* = the repo's ticket ids.

Layer	Module	What it does and the load-bearing decisions
Simulator	sim/arena.py, load.py, actions.py	Top-down gravity-free damped Pymunk space, three chambers, two slits. The T-load is one rigid body (bar 1.4 x 0.3, stem 0.3 x 1.0). Difficulty is slit width 1.8 / 1.2 / 1.1 for easy / medium / hard. Seven macro-actions (N, S, E, W, ROT+, ROT-, WAIT) as world-frame impulses at the COM with quasi-static settling; angular impulse 0.5 gives about 34 degrees per action.
Simulator	sim/feasibility.py	An A* oracle over macro-actions on the real simulator. Certified optima easy 7, medium 13, hard 13 steps; frozen budgets 18 / 33 / 33 at 2.5x optimum. Run with python -m preceptx.sim.feasibility. This is the tool that caught the infeasible ladder.
Simulator	sim/serialise.py	Three prompt forms of the same state: numeric tuples, ASCII occupancy grid (cell 0.25 so the T's 0.3-thick members stay about one cell wide and the rotate-to-clear affordance is visible), and templated natural language. Matched in task-relevant information, not strictly isomorphic - say so in the thesis.
Simulator	sim/ outcomes.py	Geodesic distance via a waypoint graph through the slit centres, plus the four candidate Y: y_binary_progress (net positive geodesic progress over the next k=3 steps, the primary), y_continuous_displacement (the unthresholded twin), y_discrete_config (chamber at window end), y_terminal_success. k is the only free knob.
Agents	agents/ graph.py, prompts.py	LangGraph loop: agent A emits a natural-language handoff, agent B picks a structured Action under guided JSON decoding, apply steps the simulator and writes one HandoffRecord. Termination is our step budget, not LangGraph recursion. PROMPT_VERSION v2 is recorded in the manifest, because a wording change is result-affecting.
Agents	agents/ channel.py	The C0 to C4 degradation ladder, touching the A-to-B message and B's observation only. C0 passthrough, C1 whitespace-token cap (default 8), C2 one-step delivery delay, C3 observation window (plus or minus 2 grid rows, message left intact), C4 seeded token dropout (0.4). C3 is the structural guard against a CPVI floor effect.

Serving	serving/ client.py, scripts/myriad/ serve.sh	LLMClient over any OpenAI-compatible endpoint. Greedy (temperature 0), fixed seed, guided_json via xgrammar. Qwen3 thinking is disabled per request and a <think> block in a message raises ServingError rather than degrading silently. serve.sh is an SGE jobscrip that echoes vLLM, torch, GPU and revision into the job log.
Data	data/schema.py, writer.py	HandoffRecord at SCHEMA_VERSION 2, carrying observation (B's delivered view, the CPVI conditioning input), state_str (A's full view, a free C3 diagnostic), the gate fields and the four Y. Append-safe atomic Parquet parts, content-hashed datasets.
Measure	measure/ featuriser.py	Frozen sentence embeddings with a content-addressed cache keyed on encoder name, revision and text. Default BAAI/bge-base-en-v1.5 at 768 dims, chosen over 1024 to curb probe overfit at pilot N. The revision is still 'main' and must be pinned to a commit SHA before the Phase-2 freeze.
Measure	measure/ pvi_cpvi.py	The estimator. Probes are cross-fitted with StratifiedGroupKFold keyed on episode_id so no episode spans train and test - a random handoff split would leak the shared trajectory and inflate CPVI. Returns per-instance scores plus mean CPVI, mean PVI, the gap, and held-out AUROC of g_cond against g_base. Also shuffled_message_cpvi, the within-condition permutation null.
Measure	measure/ twin.py, divergence.py	The prospective twin is KL(g_cond g_base) in bits, capped at 10. Its signature takes no Y at all, so the no-outcome discipline is structural rather than a convention. It is non-negative while retrospective CPVI is signed - see section 8.
Gate	gate/ statistics.py	Three target-free statistics behind one ABC where fit() may see Y and score() never can: s_info (entropy of g_cond), s_fail (probability of failure from a failure-risk probe), s_cos (message-versus-state cosine, probe-independent). Loaded by stable string key, not class name.
Gate	gate/ calibration.py	Validates each statistic against realised failure, never CPVI - the entry point has no CPVI parameter at all. Grouped out-of-fold scoring, orientation flip, threshold chosen as the most aggressive within a firing-rate budget (default 0.2), Platt-mapped ECE with reliability bins and an unreliable-below-N=200 flag.
Experiments	experiments/ sweep.py, runner.py	SweepConfig is the grid (conditions x serialisations x difficulties x seeds) plus model, channel, jitter, step and outcome configs, all of which feed sweep_hash and the manifest. run_grid executes with bounded concurrency and serialised writes, and is resumable by episode_id. The manifest records serving_substrate and endpoint_base_url.

Experiment s	experiments/ pilot.py	G1 capability (C0 success against a 0.5 floor), G2 signal (the C0-to-hard gap in both outcome and CPVI, scored on y_binary_progress), G3 groundedness (message numbers matching true state numbers, floor 0.8). Verdict ladder proceed / retune_once / fallback, with a three-seed floor before any proceed.
Experiment s	experiments/ rq1.py	The headline driver. Per-condition summaries with bootstrap CIs, Ck-versus-C0 contrasts (Cliff's delta on success and on steps, Holm or BH corrected), a mixed model of per-handoff progress with seed random intercept and episode variance component, and episode-level Baron-Kenny mediation of success through episode-mean CPVI with a bootstrapped indirect effect.
Analysis	analysis/ stats.py, figures.py, render.py	Effect sizes, BCa bootstrap, Holm and BH correction, seed sensitivity tracked on the C0-minus-hardest gap rather than on overall success, which always varies. ANALYSIS_PROTOCOL is the citable test-per-hypothesis map, frozen with Y and V. render_episode draws the arena per step; render_transcript pairs each state with the raw and delivered message and the action.

4. What the last two merges enabled, and what still blocks you

The question of what the last push bought, specifically for making progress without Myriad, has four parts - and the honest answer includes one thing it did not buy.

(1) The interim serving path needs zero code changes. ServingConfig.base_url already accepts any host. A rented GPU running your own serve.sh reproduces Myriad semantics exactly: same vLLM, same --seed, same xgrammar guided decoding, same pinned revision, different hostname only. At roughly 1.5 to 2.5 US dollars an hour for an A100-80GB and a pilot of two to five GPU-hours, the whole pilot costs less than a takeaway. Hosted APIs are the weak fallback - no revision-pinning guarantees, different structured-output APIs, opaque batching - and local Ollama would need a small adapter, because it takes a 'format' field rather than the guided_json extra body that LLMClient sends.

(2) Interim data is permanently distinguishable from cluster data. SweepManifest records serving_substrate (from the PRECEPTX_SERVING_SUBSTRATE environment variable) and endpoint_base_url, and deliberately excludes both from sweep_hash because the substrate is an environment property, not experiment identity. Set PRECEPTX_SERVING_SUBSTRATE=interim-runpod and write to a separate data root. The policy that follows: interim data informs decisions - gate verdicts, the one allowed retune, k, prompts, budgets - and headline data is regenerated on Myriad.

(3) Four diagnostics run today with no GPU and no LLM. The feasibility search, the episode renderer, the transcript dumper and the shuffled-message audit all reconstruct from persisted records or from the simulator alone. This is the class of tooling that has already caught a task-design flaw for free, at zero compute cost, before any result was

frozen.

(4) The P0 fixes mean interim pilot evidence is actually valid. This is the underrated part. Before the last merge a pilot would have run and produced misleading evidence. C3's CPVI conditioned on the sender's full state, so the floor-effect guard the whole design leans on was inert. And the seed axis was pseudo-replication: with greedy decoding, a fixed start pose and deterministic physics, two C0 episodes at seeds 0 and 7 issued identical prompts at every step, so 'seeds for power' meant nothing, the mixed model's seed random effect had no variance to absorb, and the probe saw near-duplicate rows. Seeded start-pose jitter (x in 1.2 to 2.8, y in 1.5 to 4.5, theta in minus to plus pi over 2, keyed on the seed alone so cross-condition pairing is preserved) fixed replication, probe-data diversity and G2 class balance in one change.

What it did not buy: there is still no way to run an experiment from the command line. `run_grid`, `run_pilot` and `run_rq1` are library functions. `scripts/` contains `determinism_check.py` and the SGE jobscript, and `pyproject` declares no `[project.scripts]`. Before the pilot can run you need a driver: roughly thirty lines that builds a `SweepConfig`, constructs an `LLMClient`, calls `run_grid`, loads the records, constructs a `Featuriser`, calls `run_pilot` and writes the report. This is the smallest and highest-leverage missing piece in the repo. The second gap is that the featuriser's real encoder lives in the optional `embed extra`, so `uv sync --extra dev --extra embed` is required before any pilot analysis, and it pulls `torch`.

5. Running an experiment, step by step

Signals: eight steps from a clean clone to a pilot verdict. Steps 1 to 3 need no GPU. Steps 4 to 8 need a served model, which for now means a rented GPU. Nothing here requires Myriad.

#	Step	What you do, and what to look at
1	Environment	<code>uv venv --python 3.11</code> , then <code>uv sync --extra dev --extra embed --extra viz</code> . The <code>embed extra</code> pulls <code>torch</code> and is required for real embeddings; <code>viz</code> enables figures. Verify with <code>uv run ruff check .</code> && <code>uv run mypy --strict src/preceptx</code> && <code>uv run pytest</code> .
2	Certify the task	<code>uv run python -m preceptx.sim.feasibility</code> . Prints the oracle path length and budget per difficulty. Expect easy 7, medium 13, hard 13 against budgets 18, 33, 33. If a difficulty comes back unsolved, stop - the geometry has drifted and the pilot would misdiagnose it as a model failure.
3	Write the driver	The missing thirty lines. Build a <code>SweepConfig</code> (<code>rq1_sweep</code> gives you the factorial), construct <code>LLMClient(ServingConfig(model=..., base_url=...))</code> , call <code>run_grid(sweep, client, root=...)</code> , then <code>load_records(dataset_hash, root=...)</code> , build <code>Featuriser(EncoderConfig())</code> , call <code>run_pilot(records, featuriser, dataset_hash=...)</code> and <code>write_pilot_report(...)</code> .
4	Stand up serving	Rent one GPU. Run <code>serve.sh</code> , or <code>vllm</code> serve directly, with the 8B tier at its pinned SHA. Confirm with <code>LLMClient(...).health_check()</code> returning <code>True</code> - it hits <code>/v1/models</code> and a one-token completion.

5	Smoke and read	Run five to ten episodes on C0 and easy. Dump <code>render_transcript(records)</code> and actually read it. You are checking three things: are messages grounded in real geometry, does B's action follow from A's message, and is the coordinate convention unambiguous to the model.
6	Iterate prompts	At most two or three <code>PROMPT_VERSION</code> bumps, all before the formal gate run. This is a budgeted activity, not an open loop. Freeze the final prompt at the same moment as Y and V.
7	Formal pilot grid	C0, C1 and C4 crossed with easy and hard, at three or more seeds. <code>min_seeds_for_proceed</code> is 3, and a one- or two-seed pass is LLM noise rather than a stable gradient, so the code downgrades it to <code>retune_once</code> . Set <code>PRECEPTX_SERVING_SUBSTRATE=interim-<provider></code> first. Then run <code>run_pilot</code> and read the rendered report.
8	Act on the verdict	<code>proceed</code> : freeze Y, V, encoder revision, serialisation, channel parameters and prompts, and commit <code>PREREGISTRATION.md</code> before the main sweep. <code>retune_once</code> : apply exactly one retune and re-gate. <code>fallback</code> : elevate RQ3a to the headline - it is the pre-planned fallback that can carry the dissertation alone.

On Myriad, steps 4 to 8 are identical. `qsub scripts/myriad/serve.sh`, point `base_url` at the compute node, unset the interim substrate variable, and write to a fresh data root. Re-running the pilot grid on the cluster to confirm the gates costs about an hour and inoculates the thesis against the obvious question of whether your pilot ran on different hardware.

6. What is not built, and what each piece is waiting on

Signals: eleven open tickets. Only three genuinely need experiment findings; the rest are buildable now or need a written decision rather than data. The distinction matters because it tells you where build-ahead effort pays and where it would become rework.

Ticket	What it is	Waiting on	The plan, and why it waits or does not
DSE-005	Model-ladder benchmark	A served endpoint	Throughput, time-to-first-token, peak memory, JSON-schema adherence rate and a ten-episode C0 smoke, emitted as one comparison table. Write it now and run it the hour the GPU is up; its table also feeds the Myriad allocation request. The 70B repo id, 'Gemma-4' and 'Mistral Small 4' in the roadmap ladder are unverified placeholders until this runs.

DSE-018	Runtime gate and controls	Nothing structural; the threshold waits on calibration	RuntimeGate at the A-to-B seam, a matched-firing-rate control, a random-trigger control and a mode flag. Buildable now - the record schema already carries gate_blocked, gate_retries and message_blocked. One design trap must be settled first: under greedy decoding a re-prompt is a fixed point, since the same prompt gives the same message gives the same block. The retry prompt must differ, which makes the gate-feedback template part of the treatment and something to pre-register, not an implementation detail.
DSE-021	RQ1 robustness cells	A per-role-client refactor, then RQ1 data	Heterogeneous pair plus the serialisation A/B. Blocked on a small refactor nobody has ticketed: the runner drives both roles from one client, so EpisodeRunner needs client_a and an optional client_b. Do the refactor now, run the cells after RQ1.
DSE-022	RQ2 analysis	RQ1 episodes	Twin agreement (correlation, Bland-Altman), proxy tracking (rank correlation of each statistic with CPVI, AUROC for predicting low CPVI and for predicting failure), and encoder sensitivity. Genuinely waits on data. The encoder-sensitivity check is only honest because the cache key now includes the encoder name; previously both encoders defaulted to revision 'main' and would have silently compared an encoder with itself.
DSE-023	RQ3a loaders	A written decision, not data	Who&When and MAST-Data parsed into HandoffRecord-compatible rows. Buildable today, CPU only, no simulator and no GPU. Blocked on the Y-on-logs hole described below.
DSE-024	RQ3a localisation	DSE-023 and the Y decision	Does low CPVI localise the annotated decisive step better than schema validity, mean cosine and the three published attribution methods, under both probe-transfer and probe-refit regimes, reporting both rather than silently picking one.
DSE-025	RQ3b causal arm	DSE-018 and a calibrated threshold	Runs a subset of RQ1 conditions in each of four modes and tests H6. The driver is buildable; the run waits on the threshold.
DSE-026	C5 supervisor arm	Optional	Decide only after RQ1 freezes.
DSE-027	SocialJax comparison	Optional	Formally defer. With a hard 21 September deadline it cannot pay for itself.

DSE-029	Reproducibility hardening	Partly landed	CITATION.cff and the .gitattributes LFS allowlist are in. Remaining: pin the encoder revision behind a check that fails on an unpinned one, confirm manifest completeness on a real run, and commit a rendered demo trace - the renderer that ticket assumed now exists.
DSE-030	Thesis export	Frozen results	Stable-filename figure and table export plus an auto-generated reproducibility appendix built from the manifests. Buildable now against fixtures, and it saves a scramble in September.

The one unsolved methodological hole. CPVI needs a per-handoff outcome. On the simulator, physics supplies it. On Who&When, a corpus of failure logs, trace-level Y is effectively single-class, so 'refit probes on a held-out portion of the logs' is unfittable as the methodology currently states it; and using the decisive-error annotation as Y trains the probe on the very label the localisation claim is evaluated against, which is circularity with extra steps. Three workable definitions exist: (a) a mixed-outcome corpus with trace success as Y, using MAST-Data which contains successes; (b) a local next-step Y constructed from the log itself, safe only in its annotation-free variants; (c) target-free statistics as the transfer object, since s_cos needs no Y at all and the simulator-trained probe transfers without refitting. The recommended shape is (c) for the transfer arm and (a) for the refit arm, reporting both. Write this as a one-page design note before DSE-023 is built. It will become a methodology subsection, and it is the one place RQ3a could quietly die during build week.

7. Challenges and what to expect

Signals: six live risks, each with a pre-agreed branch so that a risk firing is a decision rather than a crisis. R1 is the one that could actually end the headline.

Risk	What it looks like when it fires	The branch you take
R1 Models cannot ground 2D geometry	G1 fails: C0 self-play success sits below 0.5 on easy. Or G3 fails: messages cite coordinates that do not match the true state. This is the most likely single failure mode in the project.	One retune only - tighten prompts or switch serialisation, then re-gate. If it still fails, elevate RQ3a to the headline. Do not scramble. The thesis states this risk honestly either way; a documented negative on a well-specified task is a real chapter, not a wasted one.
R2 No measurable gradient	G2 fails: the C0-to-hard gap in success is under 0.1, or mean CPVI does not fall in the right direction.	Retune difficulty or serialisation once, then re-gate. Note the CPVI half of G2 is deliberately directional only, with a floor of 0.0, because CPVI's bit-scale is uncalibrated until the pilot runs. Set a pre-registered positive floor once the pilot reveals the scale.

R3 CPVI floor effect	Mean CPVI near zero across conditions because the shared state already predicts the outcome.	C3 is the structural guard and now works correctly after the P0-1 fix. Report the PVI minus CPVI gap as a finding in its own right: a large gap is the quantitative statement that most apparent message value was an echo of state.
R4 Probe overfit at pilot N	auroc_train_cond far above held-out AUROC. Concatenated 1536-dim features against a few hundred handoffs invites this.	The overfit monitor is already reported. Pre-register how C and the logistic-versus-MLP choice get made on pilot data only, then freeze. Leaving the probe-selection procedure implicit is exactly where researcher-degrees-of-freedom accusations land.
R5 'CPVI is just word count'	The first sceptical question anyone will ask. C1 manipulates length directly, so the confound is structural rather than incidental.	Pre-register a length-controlled analysis: partial Spearman of CPVI with outcome given message length, and length as a covariate in path b of the mediation. C1 then becomes the demonstration that construct and confound dissociate.
R6 Compute scarcity	The Myriad allocation lands late or small.	Default to the 14B workhorse, which fits every GPU in the envelope. The pilot on interim compute is what converts the eventual window into headline sweeps rather than debugging.

Expected scale, so the compute ask is grounded. For the headline episode-success contrast at alpha 0.05 with Holm over four contrasts and 80 per cent power: a C0-to-hard gap of 0.4 needs roughly 25 to 30 episodes per condition, a gap of 0.3 needs about 45 to 50, a gap of 0.2 needs 90 to 100. With jitter times seeds supplying true replicates, ten jittered instances by five seeds gives 50 episodes per condition, so about 250 episodes across C0 to C4, roughly three to six thousand steps, roughly six to twelve thousand LLM calls - single-digit GPU-hours on a batched 14B. The pilot's measured gap picks the row, and the compute request follows from it rather than being guessed.

8. Signals and expectations for the thesis draft

8.1 What each experiment emits, and what a result looks like

Signals: six hypotheses, each with a named artefact, an expected shape and a reportable null. Draft the methodology subsections against this table; it is what keeps the thesis and the code in the same universe.

H	Test	Expected positive shape	What a null means, and how to report it
---	------	-------------------------	---

H1	Success and efficiency fall as the channel degrades, C0 to C4	Monotone decline in episode success rate with non-overlapping bootstrap CIs; Cliff's delta on steps-to-goal positive for each Ck against C0, Holm-corrected. Artefacts: rq1.json, the results CSV, the outcome-versus-condition figure.	A flat gradient means the task does not discriminate the channel. Report it as a task-design finding rather than a measurement failure, and check whether the pilot's G2 verdict was already telling you this.
H2	CPVI tracks the degradation and explains outcome variance beyond shared state	Mean CPVI falls across conditions; episode-level Baron-Kenny mediation gives a non-zero indirect effect a times b with a bootstrap CI excluding zero, and a shrinking direct path. Artefacts: the mediation block in rq1.json, scores.parquet.	If CPVI does not mediate, the honest reading is that the channel effect runs through something other than usable information - which is the Eccles absent-versus-unused distinction and is itself reportable.
H3	Retrospective and prospective CPVI agree	High Pearson and Spearman correlation, near-zero Bland-Altman bias, tight limits of agreement. Artefact: TwinAgreement.	Expect structural attenuation wherever messages mislead. The prospective twin is a KL divergence and so non-negative, while retrospective CPVI is signed. Pre-register agreement stratified by retrospective sign, and frame the one-sidedness as a finding about the limits of runtime measurement.
H4	A cheap target-free statistic tracks CPVI closely enough to gate on	Rank correlation of each statistic with CPVI, plus AUROC of each for predicting low CPVI and for predicting failure. s_cos matters most because it is probe-independent. Artefact: the RQ2 report.	If only probe-dependent statistics track, say so plainly. It bounds the gate's deployability and is a real limitation to state rather than bury.
H5	Boundary CPVI localises the decisive step in real failure logs	Beat schema validity, mean embedding cosine and the published methods whose best step accuracy is 14.2 per cent. Report the transfer and refit regimes separately. Artefacts: the RQ3a comparison table and a human-agreement kappa on a sample.	The headroom is genuinely large, so a modest positive is publishable. A null needs the Y-on-logs construction to be defensible first, or it is uninterpretable.

H6	Gating low-information handoffs causally improves outcomes	Gate-active beats both the matched-firing-rate control and the random-trigger control on success and efficiency, with effect sizes and intervals. Report firing rate and retry counts alongside.	A clean null is a legitimate result and should be presented as one. Also acknowledge threshold transport: the calibrated threshold is applied in runs whose message distribution the gate itself perturbs downstream.
----	--	--	---

8.2 Numbers to have at hand while drafting

Task and geometry: slit ladder 1.8 / 1.2 / 1.1; T bar 1.4 x 0.3 and stem 0.3 x 1.0, giving a vertical extent of 1.3; the threading threshold is the shorter member, the stem at 1.0, and it is invariant to rotation; oracle optima 7 / 13 / 13; budgets 18 / 33 / 33; rotation about 34 degrees per action. **Measurement:** encoder bge-base-en-v1.5 at 768 dims; probe L2 logistic with $C = 1.0$; label horizon $k = 3$; C1 cap 8 whitespace tokens; C3 window plus or minus 2 rows; C4 dropout 0.4; gate firing-rate budget 0.2; ECE flagged unreliable below $N = 200$. **Literature anchors:** MAST failure rates 41 to 86.7 per cent across 1,600 or more traces and seven frameworks at $\kappa 0.88$; Who&When best baselines 53.5 per cent on the responsible agent and 14.2 per cent on the decisive step.

8.3 Where the current lit-review text and the code disagree

Signals: the methodology chapter you are drafting in parallel currently states several things the code does not do. Each row is either a text fix or a code fix, and each decision is cheap now and forbidden after the Y and V freeze. This is the most directly actionable section for the parallel drafting.

Where	The text says	The code does, and the call
Conditioning state, §9.2	Conditioned on the shared state that both agents observe.	Conditions on the receiver-observed state; under C3 that is the windowed view. Fix the text, and add the caveat that C3's CPVI sits against a smaller baseline by design, so cross-condition comparisons involving C3 are against the receiver's own baseline rather than same-baseline magnitudes.
Prospective twin, §9.4	The same fitted conditional probe applied at the handoff to produce a predictive distribution.	Per-instance $KL(g_cond g_base)$ in bits. Fix the text: state the KL form, the interpretation as expected CPVI under the probe's own belief, and the one-sidedness.

Runtime statistics, abstract versus §9.5	The abstract names failure-risk, JSD and cosine; §9.5 names entropy, failure-risk and cosine.	The code implements entropy, failure-risk and cosine, with JSD as the RQ2 bridge. Three-way drift. Fix the abstract to entropy and keep JSD as the reported bridge.
Continuous probe, §9.3	A heteroscedastic linear or shallow-network regressor.	Homoscedastic, with the deviation recorded in ProbeConfig.variance_model and the heteroscedastic path raising NotImplementedError so the deviation is auditable rather than silent. Default: amend the text, and implement only if the pilot's continuous twin misbehaves.
C4, §9.2	Token dropout or paraphrase corruption.	Dropout only. Fix the text - a paraphrase channel needs a model call inside the channel, which breaks the requirement that the channel is cheap and deterministic.
Agreement round	The two agents must agree a single macro-action.	B decides alone after reading A's message. Fix the text to 'B, informed by A's handoff, selects the joint action'. An acknowledgement round would create a second boundary and muddy the unit of measurement.
Outcome, abstract	Whether the joint action after a handoff makes net progress.	Net progress over the next $k = 3$ steps. Fix the text once k freezes, or set $k = 1$ as primary if the pilot supports it - $k = 1$ is the abstract's literal reading.
Serialisation	The three forms are isomorphic in information.	Approximately matched: the grid quantises pose to 0.25 cells and shows angle only through the footprint. Soften to 'matched in task-relevant information up to representation precision'.

Difficulty rationale	The T must rotate to pass a narrow slit.	Geometrically false, and now corrected in code: rotation only swaps which member jams, so the tightest threadable slit is the shorter member's length and is rotation-invariant. Difficulty is graded by threading clearance above the 1.0 stem threshold. State this plainly in the methodology, because a reviewer will otherwise ask why rotation is the difficulty knob.
----------------------	--	--

9. The next fourteen days

Signals: ordered by what unblocks the most. Items 1 to 3 need no GPU and no decisions from anyone else. Item 4 is the one that converts the project from built to running.

#	Do	Why it sits in this position
1	Write the pilot driver and PREREGISTRATION.md v0	The driver is about thirty lines and is the literal blocker between a served model and a pilot verdict. The pre-registration skeleton costs an afternoon and is what separates a confirmatory result from a fished one: Y and k, conditioning semantics, V and its selection rule, encoder and revision, serialisation, the C1, C3 and C4 parameters, jitter and seed count, budgets, the corrected analysis protocol, the gate feedback template, and the G1, G2 and G3 thresholds.
2	Write the RQ3a Y-on-logs design note	One page. It unblocks DSE-023 and DSE-024, which is the fallback that can carry the dissertation alone - and a fallback you have not verified is not a fallback. Pair it with a timeboxed loader spike: download Who&When and MAST, parse ten traces each into HandoffRecord-compatible rows, count extractable handoffs, and confirm which corpus has enough successful traces for the refit arm.
3	Pin the encoder revision, and do the per-role-client refactor	The encoder revision is still 'main', which means a moving target under a manifest that claims to pin everything, and a result with an unrecorded revision is not a result by your own rules. The refactor, EpisodeRunner taking client_a and an optional client_b, unblocks DSE-021 and the C5 stub, and is far cheaper now than after more call sites accrete.
4	Rent a GPU, run DSE-005, then the pilot	This is the step that changes the project's state. Serve the 8B at its pinned SHA, run the benchmark table, smoke ten episodes and read the transcripts, iterate prompts at most three times, then run the formal grid and run_pilot. Label everything interim-<provider>.

5	Act on the verdict and freeze	proceed means freeze Y, V, encoder, serialisation, channel parameters and prompts, commit PREREGISTRATION.md v1, and start the RQ1 sweep the moment compute allows. Draft chapters 3 and 4 in the same week while the system is fresh - the roadmap's own writing rule, and the one most often broken.
---	-------------------------------	--

Two cut-lines to exercise now rather than later. Formally defer DSE-027, the SocialJax comparison: with a hard 21 September deadline and a paper that reuses RQ1 and RQ2, it cannot pay for itself. Decide DSE-026, the C5 supervisor arm, only after RQ1 freezes. Annotating both issues today makes the backlog reflect the real plan, which matters more than it sounds when you are reading it under time pressure in six weeks.

One last honest note on the clock. By the roadmap calendar this is week 4, which is Phase 2 to 3 territory, and Phase 1 has not run. The saving grace is that the code for Phases 1 to 4 is built, so the science phases compress hard once serving exists. The risk is not that the remaining work is large; it is that the pilot keeps slipping behind another week of building. The engineering has stopped being the constraint.